

AuditPlatform

Rapport d'analyse — Mon ProjetIAGitLabb

Généré le 06/05/2026 à 17:15

■ Informations du projet

Nom du projet	Mon ProjetIAGitLabb
URL du projet	eyabenrached17/mon-projetIAGitLabb
Branche analysée	main
Date de l'analyse	04/05/2026 16:45
Modèle LLM	llama-3.3-70b-versatile
Analyse OWASP	Activée

■ Scores

Qualité	20	20/100
Sécurité	20	20/100
Performance	20	20/100

■ ■ Vulnérabilités (36)

A03 — INJECTION
Sévérité : CRITIQUE
Fichier : app.py — ligne 11
Correction : Utiliser des requêtes préparées pour éviter les injections SQL, par exemple : cursor.execute('SELECT * FROM users WHERE username = ?', (username,))
A02 — CRYPTOGRAPHIC FAILURES
Sévérité : CRITIQUE

Fichier : app.py — ligne 17

Correction : Hasher les mots de passe avec un algorithme sécurisé comme bcrypt, Argon2 ou PBKDF2

SECRETS ET DONNÉES SENSIBLES

Sévérité : CRITIQUE

Fichier : app.py — ligne 21

Correction : Enregistrer les clés secrètes et les API keys en tant que variables d'environnement ou dans un fichier de configuration sécurisé

A03 — INJECTION

Sévérité : CRITIQUE

Fichier : app.py — ligne 30

Correction : Utiliser la fonction subprocess avec des arguments séparés pour éviter les injections de commandes, par exemple : `subprocess.run(['ping', host])`

Complexité

Sévérité : HAUTE

Fichier : app.py — ligne 11

Correction : Utiliser des requêtes préparées pour éviter les injections SQL.

Maintenabilité

Sévérité : HAUTE

Fichier : app.py — ligne 17

Correction : Stockez les mots de passe de manière sécurisée en utilisant un algorithme de hachage fort comme bcrypt ou Argon2.

Maintenabilité

Sévérité : HAUTE

Fichier : app.py — ligne 21

Correction : Évitez les clés secrètes hardcodées et utilisez des variables d'environnement ou un système de gestion de secrets.

Maintenabilité

Sévérité : HAUTE

Fichier : app.py — ligne 25

Correction : Remplacez l'algorithme de hachage MD5 par un algorithme plus sécurisé comme bcrypt, Argon2 ou PBKDF2.

Maintenabilité

Sévérité : HAUTE

Fichier : app.py — ligne 30

Correction : Évitez les commandes shell non sécurisées et utilisez des bibliothèques Python pour les opérations réseau.

A02 — CRYPTOGRAPHIC FAILURES

Sévérité : HAUTE

Fichier : app.py — ligne 25

Correction : Utiliser un algorithme de hachage plus sécurisé que MD5, comme SHA-256 ou bcrypt

N+1

Sévérité : HAUTE

Fichier : app.py — ligne 12

Correction : Utiliser une requête SQL avec des jointures pour éviter les requêtes N+1. Gain estimé : 50% de réduction de la charge sur la base de données.

Ressource

Sévérité : HAUTE

Fichier : app.py — ligne 30

Correction : Utiliser une commande shell sécurisée pour éviter les injections de commandes. Gain estimé : 20% de réduction du risque de sécurité.

Gestion erreurs

Sévérité : HAUTE

Fichier : app.py — ligne 11

Correction : Utiliser des requêtes préparées pour éviter les injections SQL

Gestion erreurs

Sévérité : HAUTE

Fichier : app.py — ligne 17

Correction : Stocker les mots de passe de manière sécurisée en utilisant un algorithme de hachage fort comme bcrypt ou Argon2

Gestion erreurs

Sévérité : HAUTE

Fichier : app.py — ligne 21

Correction : Utiliser des clés secrètes générées de manière sécurisée et stockées en dehors du code source

Gestion erreurs

Sévérité : HAUTE

Fichier : app.py — ligne 25

Correction : Utiliser un algorithme de hachage fort comme bcrypt ou Argon2 pour hacher les mots de passe

Gestion erreurs

Sévérité : HAUTE

Fichier : app.py — ligne 30

Correction : Utiliser des commandes shell sécurisées pour éviter les injections de commandes

Lisibilité

Sévérité : MOYENNE

Fichier : app.py — ligne 10

Correction : Nommez les fonctions de manière plus descriptive pour améliorer la lisibilité du code.

SELECT*

Sévérité : MOYENNE

Fichier : app.py — ligne 11

Correction : Utiliser une requête SQL avec des champs spécifiques au lieu de SELECT * pour réduire la quantité de données transférées. Gain estimé : 10% de réduction de la charge sur la base de données.

Boucle O(n²)

Sévérité : MOYENNE

Fichier : app.py — ligne 25

Correction : Utiliser un algorithme de hachage plus rapide pour réduire la charge de calcul. Gain estimé : 15% de réduction de la charge de calcul.

Docstring manquant

Sévérité : MOYENNE

Fichier : app.py — ligne 10

Correction : `"""Récupère un utilisateur par son nom d'utilisateur."""`

Docstring manquant

Sévérité : MOYENNE

Fichier : app.py — ligne 16

Correction : `"""Crée un nouvel utilisateur avec le nom d'utilisateur et le mot de passe donnés."""`

Docstring manquant

Sévérité : MOYENNE

Fichier : app.py — ligne 25

Correction : `"""Hache le mot de passe donné."""`

Docstring manquant

Sévérité : MOYENNE

Fichier : app.py — ligne 29

Correction : `"""Envoie une requête ping à l'hôte donné."""`

Docstring manquant

Sévérité : MOYENNE

Fichier : app.py — ligne 33

Correction : ""Fonction principale du programme.""

Nommage ambigu

Sévérité : MOYENNE

Fichier : app.py — ligne 21

Correction : Utiliser un nom de variable plus descriptif pour la clé secrète

SOLID-D

Sévérité : MOYENNE

Fichier : app.py — ligne 6

Correction : Utiliser l'injection de dépendances pour gérer la connexion à la base de données

SOLID-S

Sévérité : MOYENNE

Fichier : app.py — ligne 10

Correction : Diviser la fonction get_user en plusieurs fonctions plus petites et plus spécialisées

SOLID-S

Sévérité : MOYENNE

Fichier : app.py — ligne 16

Correction : Diviser la fonction create_user en plusieurs fonctions plus petites et plus spécialisées

Docstring manquant

Sévérité : FAIBLE

Fichier : app.py — ligne 1

Correction : Ajoutez des docstrings pour expliquer le but et l'utilisation de chaque fonction.

Commentaire absent

Sévérité : FAIBLE

Fichier : app.py — ligne 6

Correction : Ajoutez des commentaires pour expliquer les sections complexes du code.

Mémoire

Sévérité : FAIBLE

Fichier : app.py — ligne 17

Correction : Fermer la connexion à la base de données après utilisation pour éviter les fuites de mémoire. Gain estimé : 5% de réduction de la consommation de mémoire.

Commentaire absent

Sévérité : FAIBLE

Fichier : app.py — ligne 11

Correction : # Construit la requête SQL pour récupérer l'utilisateur

Commentaire absent

Sévérité : FAIBLE

Fichier : app.py — ligne 17

Correction : # Insère le nouvel utilisateur dans la base de données

Commentaire absent

Sévérité : FAIBLE

Fichier : app.py — ligne 26

Correction : # Récupère le hachage MD5 du mot de passe

Commentaire absent

Sévérité : FAIBLE

Fichier : app.py — ligne 30

Correction : # Exécute la commande ping sur l'hôte donné

■ Recommandations

1. Utilisation de requêtes préparées

Les requêtes préparées permettent d'éviter les injections SQL en séparant le code SQL des données. Exemple : `cursor.execute('SELECT * FROM users WHERE username = ?', (username,))`

2. Stockage sécurisé des mots de passe

Utilisez un algorithme de hachage fort comme bcrypt ou Argon2 pour stocker les mots de passe de manière sécurisée. Exemple : `hashed_password = bcrypt.hashpw(password.encode(), bcrypt.gensalt())`

3. Gestion des clés secrètes

Évitez les clés secrètes hardcodées et utilisez des variables d'environnement ou un système de gestion de secrets. Exemple : `SECRET_KEY = os.environ['SECRET_KEY']`

4. Utilisation d'algorithmes de hachage sécurisés

Remplacez l'algorithme de hachage MD5 par un algorithme plus sécurisé comme bcrypt, Argon2 ou PBKDF2. Exemple : `hashed_password = bcrypt.hashpw(password.encode(), bcrypt.gensalt())`

5. Évitement des commandes shell non sécurisées

Évitez les commandes shell non sécurisées et utilisez des bibliothèques Python pour les opérations réseau. Exemple : `import subprocess; subprocess.run(['ping', host])`

6. Utiliser des requêtes préparées

Les requêtes préparées permettent d'éviter les injections SQL en séparant le code SQL des données. Par exemple, au lieu de concaténer les variables dans la requête, utilisez des requêtes préparées comme `cursor.execute('SELECT * FROM users WHERE username = ?', (username,))`

7. Hasher les mots de passe

Il est essentiel de hasher les mots de passe pour protéger les informations sensibles des utilisateurs. Utilisez un algorithme de hachage sécurisé comme bcrypt, Argon2 ou PBKDF2 pour stocker les mots de passe de manière sécurisée

8. Protéger les clés secrètes

Les clés secrètes et les API keys doivent être protégées pour éviter les accès non autorisés. Enregistrez-les en tant que variables d'environnement ou dans un fichier de configuration sécurisé pour les garder en sécurité

9. Utiliser des algorithmes de hachage sécurisés

Les algorithmes de hachage comme MD5 sont considérés comme faibles et doivent être remplacés par des algorithmes plus sécurisés comme SHA-256 ou bcrypt pour protéger les données sensibles

10. Éviter les injections de commandes

Les injections de commandes peuvent être évitées en utilisant la fonction subprocess avec des arguments séparés. Par exemple, au lieu de concaténer les variables dans la commande, utilisez `subprocess.run(['ping', host])` pour exécuter la commande de manière sécurisée

11. Optimisation des requêtes SQL

Utiliser des requêtes SQL avec des champs spécifiques et des jointures pour réduire la quantité de données transférées et la charge sur la base de données. Exemple : `SELECT id, username FROM users WHERE username = 'admin'` au lieu de `SELECT * FROM users WHERE username = 'admin'`.

12. Fermeture des connexions à la base de données

Fermer les connexions à la base de données après utilisation pour éviter les fuites de mémoire. Exemple : `conn.close()` après avoir terminé d'utiliser la connexion.

13. Utilisation de commandes shell sécurisées

Utiliser des commandes shell sécurisées pour éviter les injections de commandes. Exemple : utiliser la fonction `subprocess.run()` au lieu de `os.system()` pour exécuter des commandes shell.

14. Ajouter des docstrings

Ajouter des docstrings pour expliquer le but de chaque fonction et méthode

15. Utiliser des noms de variables descriptifs

Utiliser des noms de variables qui reflètent leur contenu et leur utilisation

16. Ajouter des commentaires

Ajouter des commentaires pour expliquer les parties complexes du code

17. Stockez les mots de passe de manière sécurisée

Utilisez un algorithme de hachage fort comme `bcrypt` ou `Argon2` pour stocker les mots de passe. Par exemple, vous pouvez utiliser la bibliothèque ``bcrypt`` pour hacher les mots de passe : ``hashed_password = bcrypt.hashpw(password.encode(), bcrypt.gensalt())``

18. Utilisez des clés secrètes générées de manière sécurisée

Utilisez des clés secrètes générées de manière sécurisée et stockées en dehors du code source. Par exemple, vous pouvez utiliser la commande ``openssl rand -base64 32`` pour générer une clé secrète